SCRIPT-CONTROLLED ACL – RESTRICT RECORD ACCESS BASED ON FIELD VALUE

PHASE 3: PROJECT DESIGN PHASE

The Project Design Phase focused on designing the ACL structure and access-control process in ServiceNow. The required table, fields, conditions, and script logic were planned to provide secure and controlled access to records. The design was prepared based on the identified requirements before moving to the development phase.

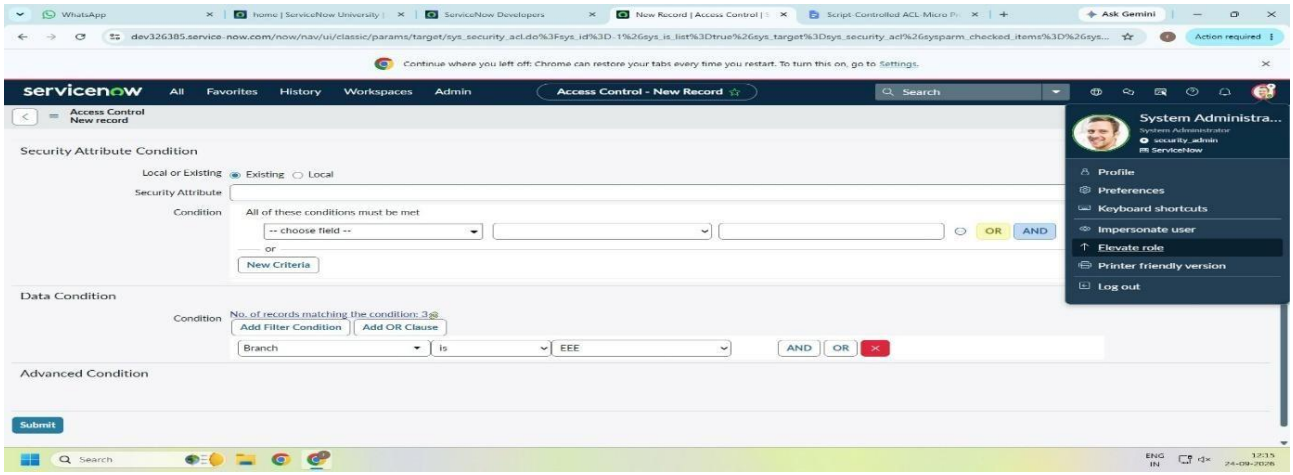
The design phase converts the requirements into the actual structure of the ACL. It fixes what type of rule is needed, which operation it protects, which conditions it uses and how the script decides the outcome. Designing before configuring avoids mistakes such as protecting the wrong operation or forgetting the administrator.

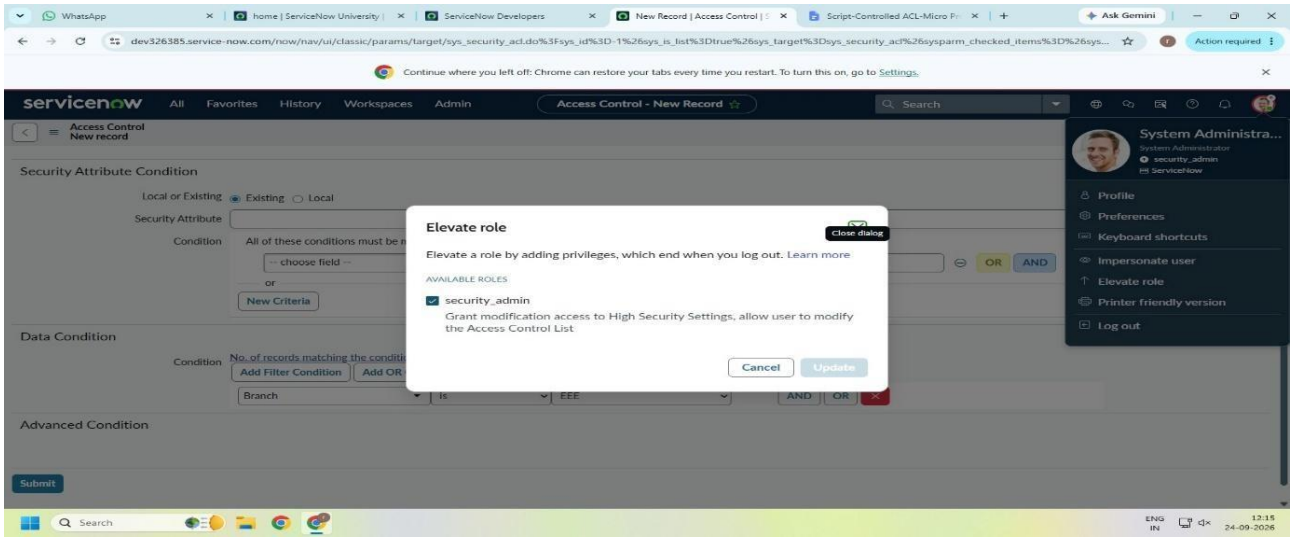
3.1 Designing the ACL Structure

The structure of the ACL was designed based on the identified access requirements. The required access rules and permissions were planned for implementation.

A new Access Control record is created from the navigator. The form is divided into sections: the general settings (type, operation, decision type, name), the **Requires role** list, the **Security Attribute Condition**, the **Data Condition** and the **Advanced** section with the script.

In the first screenshot the Data Condition already shows *Branch is EEE*, which limits the ACL to the EEE records. The second screenshot shows the **Elevate role** dialog. Creating or editing an ACL is a protected action, so the logged-in system administrator must first elevate to the *security_admin* role, which grants modification access to high-security settings such as the Access Control List.



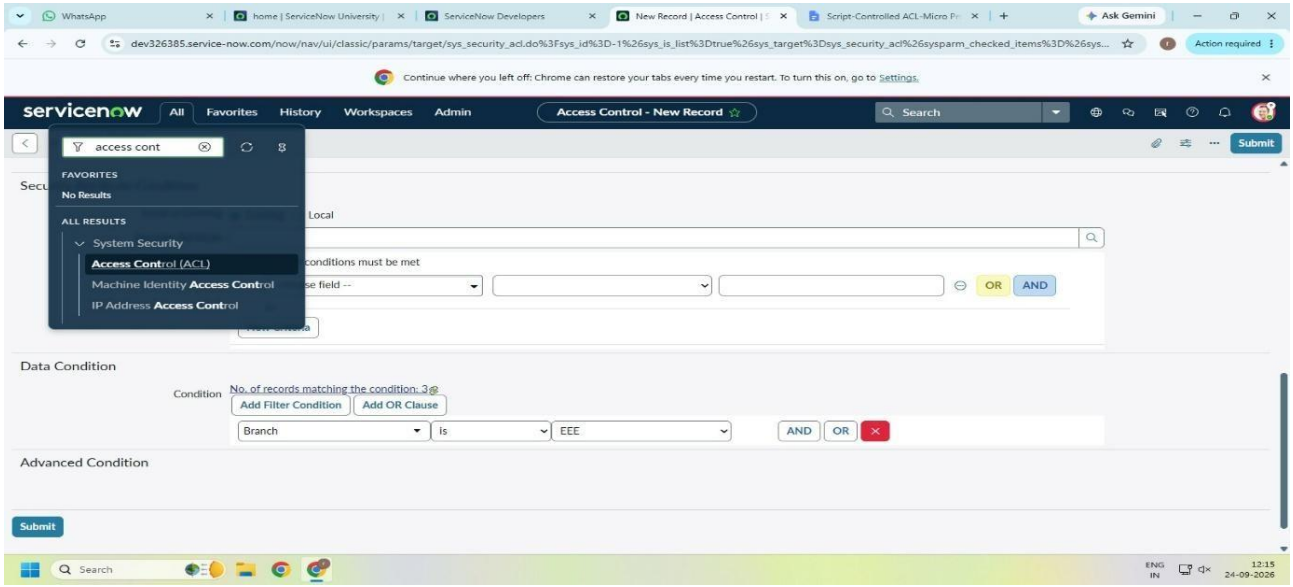


3.2 Configuring ACL Parameters

The necessary ACL parameters were identified for configuring the access control. The required table, operation, and access settings were considered.

The main parameters are the following. **Type** is *record*, so the rule protects records of a table. **Operation** defines what is protected (read, create, write, delete). **Decision Type** is *Allow If*, meaning access is granted only when all the conditions are met. **Name** is the table, Institution Details [u_institution_details], and the field can be left as None to protect the whole record.

The screenshot shows how the Access Control module is found by typing “access control” in the navigator filter, under System Security. The *Admin overrides* and *Active* options are also visible on this form.

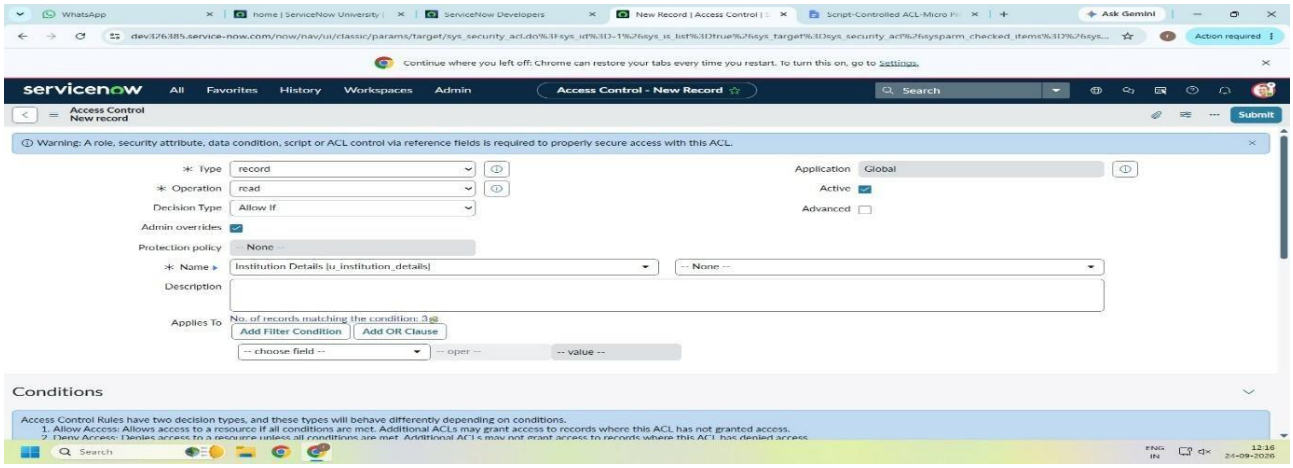


3.3 Defining Conditions

The conditions required for controlling record access were defined. These conditions help determine whether access should be allowed or restricted.

ServiceNow explains the two decision types on the form itself. With *Allow If*, access is granted when all conditions are met, and other ACLs may still grant access where this one has not. With *Deny Unless*, access is refused unless all the conditions are met, and other ACLs cannot grant access to a record that this ACL has denied. This project uses Allow If because it is easier to reason about: the ACL grants access explicitly to the intended users.

The screenshot shows the ACL form with the type set to record, the operation set to read, the decision type set to Allow If, and the table selected. The *Applies To* field can further narrow the records the ACL covers.



3.4 Designing the Access Control Logic

The access-control logic was designed using the identified conditions and requirements. The logic provides a clear approach for controlling access to specific records.

The logic follows a simple order of checks. First, administrators are always allowed. Second, users who hold the branch role are allowed to see the EEE records. Third, anything else is denied by returning false. Placing the deny at the end guarantees that no user can get access by accident.

The screenshot shows the Advanced Condition section, where the option for a script is enabled and ECMAScript 2021 (ES12) mode is turned on. The script that was designed is shown below.

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }
    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')) {
        return true;
    }
    // Deny access for all others
    return false;
})();
```

